Интерфейсы
List
Set
Queue



А зачем вообще нужны коллекции?

IThub

Представим, что нам нужно хранить имена студентов. Самый простой вариант:

```
String student1 = "Alex";
String student2 = "Maria";
String student3 = "John";
```

Но что делать, если студентов стало 100? Или 10 000?

Можно использовать массив:

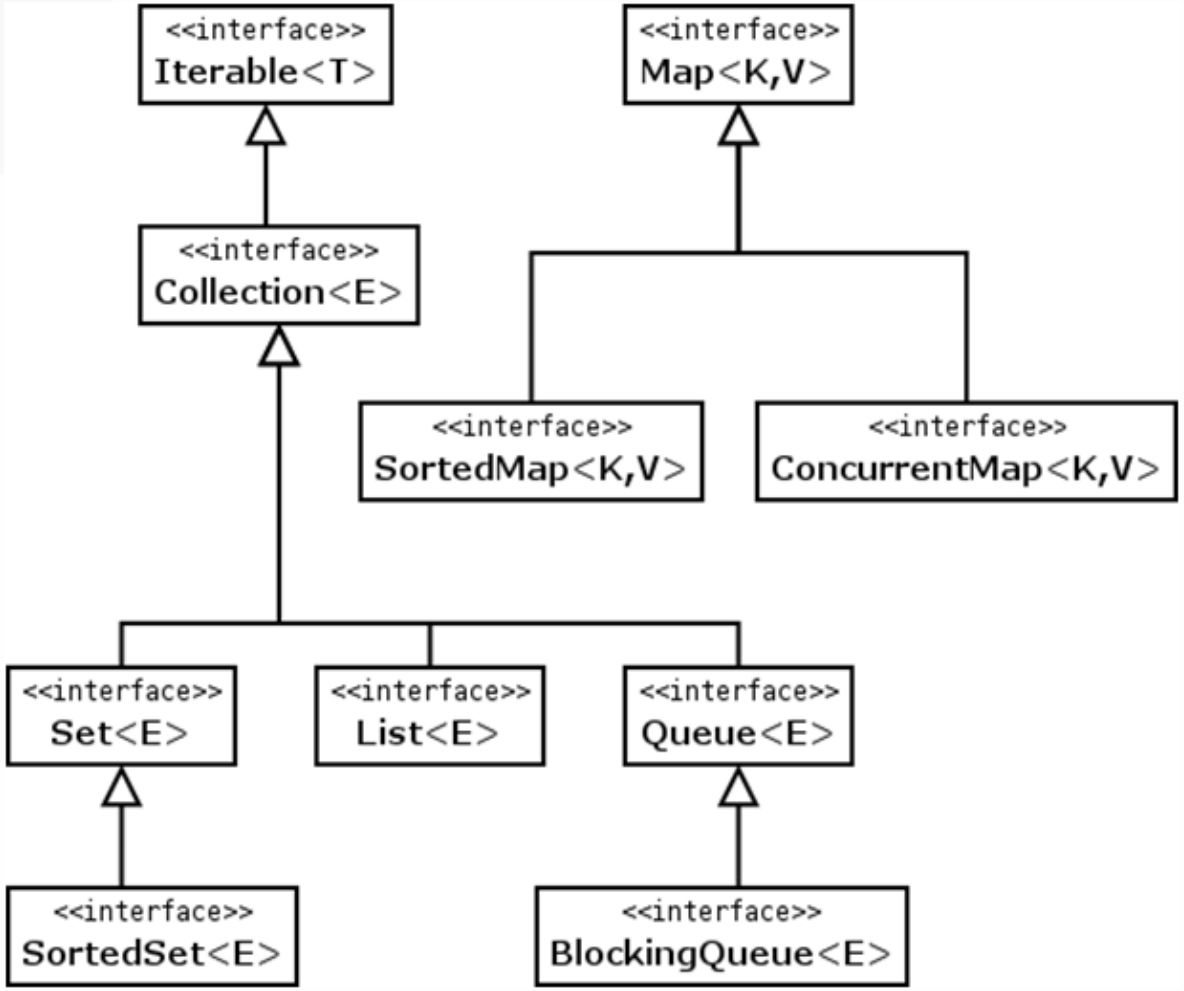
```
String[] students = new String[10000];
```

Но массив имеет ограничения:

- размер задаётся заранее;
- неудобно добавлять и удалять элементы;
- нет готовых операций поиска, сортировки и т.д.

Для этого и существуют коллекции

java.util Collections Framework



Java Collections Framework (JCF) - набор интерфейсов и классов для работы с группами объектов.

Главный вопрос: какую коллекцию выбрать для конкретной задачи?

Интерфейс Collection

IThub

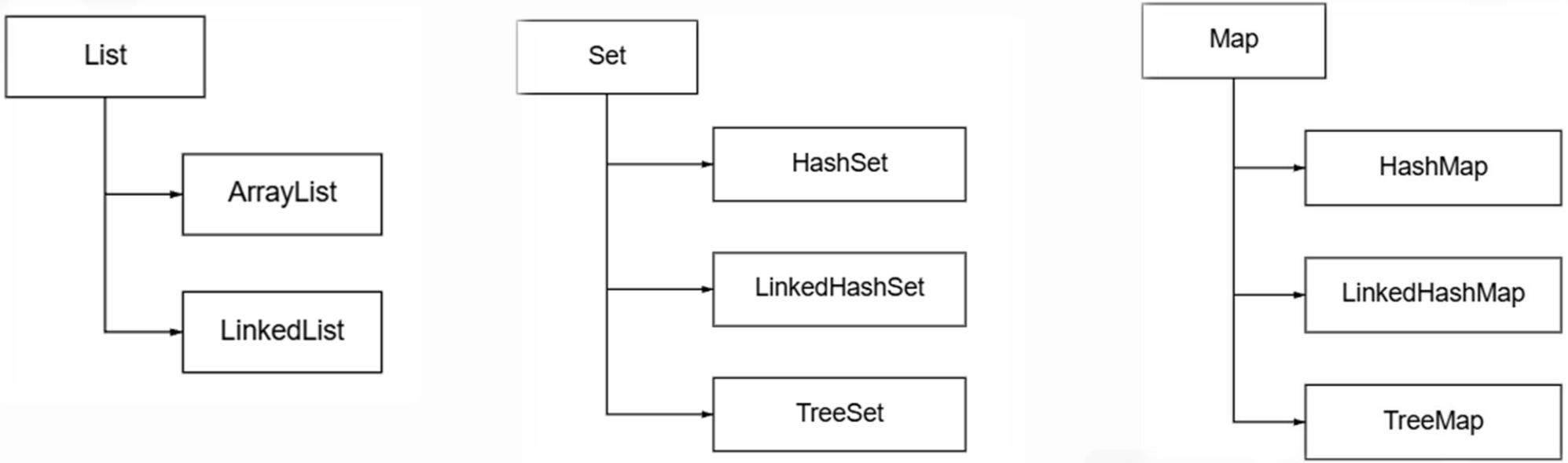
```
interface Collection<E> extends Iterable<E>
```

- основа, на которой построена инфраструктура Collections Framework, т.к. он должен быть реализован любым классом, определяющим коллекцию
- расширяет Iterable (можно проходить в цикле стиля for-each)

Главная идея Collections Framework

IThub

В java collections framework есть интерфейсы и реализации:



Интерфейс отвечает на вопрос: что умеет коллекция?

Реализация отвечает на вопрос: как именно это работает внутри?

Java Collections Framework

Структура интерфейсов и классов

IThub

Iterable<E>
(интерфейс)
+ iterator()

Collection<E>
(интерфейс)
+ size() + isEmpty() + add(E) + remove(Object) + iterator() ...

List

Упорядоченная коллекция, допускает дубликаты элементов.

List<E>
(интерфейс)
+ get(int) + set(int, E)
+ add(int, E) + remove(int) ...

ArrayList<E>
(класс)
Реализация на основе массива. Быстрый доступ по индексу.

LinkedList<E>
(класс)
Реализация на основе связного списка. Быстрые вставки/удаления.

Vector<E>
(класс)
Устаревшая синхронизированная реализация.

Set

Коллекция уникальных элементов (дубликаты не допускаются).

Set<E>
(интерфейс)
+ size() + isEmpty() + add(E)
+ remove(Object) ...

HashSet<E>
(класс)
Реализация на основе хеш-таблицы. Не гарантирует порядок.

LinkedHashSet<E>
(класс)
Сохраняет порядок вставки (на основе HashSet).

TreeSet<E>
(класс)
Реализация на основе красно-черного дерева. Упорядоченные элементы.

Map

Коллекция пар «ключ → значение».
Ключи уникальны.

Map<K, V>
(интерфейс)
+ size() + isEmpty() + put(K, V)
+ get(Object) + remove(Object)
+ keySet() + values()
+ entrySet() ...

HashMap<K, V>
(класс)
Реализация на основе хеш-таблицы. Не гарантирует порядок.

LinkedHashMap<K, V>
(класс)
Сохраняет порядок вставки (на основе HashMap).

TreeMap<K, V>
(класс)
Упорядочение по ключам (на основе красно-черного дерева).

Дополнительные утилиты

Collections
(класс, утилитный)
Статические методы для работы с коллекциями (sort, reverse, shuffle и др.).

Arrays
(класс, утилитный)
Статические методы для работы с массивами (asList, sort, binarySearch и др.).

Интерфейсы, связанные с очередями

Queue<E>
(интерфейс)
+ offer(E) + poll() + peek() ...

PriorityQueue<E>
(класс)
Очередь с приоритетами (на основе кучи).

Deque<E>
(интерфейс)
Двусторонняя очередь (может работать и как стек, и как очередь).

ArrayDeque<E>
(класс)
Реализация на основе массива.

LinkedList<E>
(класс)
Реализация на основе связного списка.

Исключения

- **UnsupportedOperationException** - происходит, если коллекцию изменять нельзя
- **ClassCastException** - генерируется, когда один объект несовместим с другим (например при попытке добавления в коллекцию несовместимого объекта)
- **NullPointerException** - возникает, если предпринимается попытка сохранить объект null, а элементы null в коллекции не разрешены
- **IllegalArgumentException** - генерируется в случае применения недопустимого аргумента
- **IllegalStateException** - происходит при попытке добавления элемента в заполненную коллекцию фиксированной длины

Интерфейс List

IThub

List — упорядоченная коллекция

Особенности:

- сохраняет порядок добавления;
- имеет индексы;
- допускает дубликаты.

Полезные методы

Модификатор и тип	Метод	Описание
void	add(int index, E element)	Вставляет указанный элемент в указанную позицию этого списка. Ранее существовавшие элементы в точке вставки или после нее сдвигаются вправо. В итоге элементы списка не перезаписываются
E	get(int index)	Возвращает элемент в указанной позиции этого списка
int	indexOf(Object o)	Возвращает индекс первого вхождения указанного элемента в этот список или -1, если список не содержит элемента
E	set(int index, E element)	Присваивает element значение по индексу, указанному в index внутри вызывающего списка. Возвращает старое значение
boolean	remove(Object o)	Удаляет первое вхождение указанного элемента из этого списка, если оно присутствует
boolean	isEmpty()	Возвращает true, если эта коллекция не содержит элементов
boolean	contains(Object o)	Возвращает true, если эта коллекция содержит указанный элемент
void	clear()	Удаляет все элементы из данной коллекции

LinkedList

```
class LinkedList<E> extends AbstractSequentialList<E> implements List<E>, Deque<E>
```

- предоставляет структуру данных типа двусвязного списка

! эта реализация **не синхронизирована** (если несколько потоков одновременно обращаются к связному списку, и хотя бы один из них изменяет его структуру, необходимо выполнить внешнюю синхронизацию)

```java
public void linkedListDemo() {
    LinkedList<String> list = new LinkedList<>();
    list.add("Monday");
    list.add("Tuesday");
    list.add("Wednesday");
    list.add("Thursday");
    list.add("Friday");
    list.add("Saturday");
    list.add("Sunday");

    System.out.println("LinkedList elements size = " + list.size() + ". Elements:");
    list.forEach(System.out::println);

    System.out.println();
    boolean result = list.remove("lalala");
    System.out.println("Remove \"lalala\" result = " + result);
    result = list.remove("Wednesday");
    System.out.println("Remove \"Wednesday\" result = " + result);

    System.out.println("LinkedList elements size after remove = " + list.size() + ". Elements:");
    list.forEach(System.out::println);

    System.out.println();
    list.removeFirst();
    System.out.println(list.getFirst());
}
```

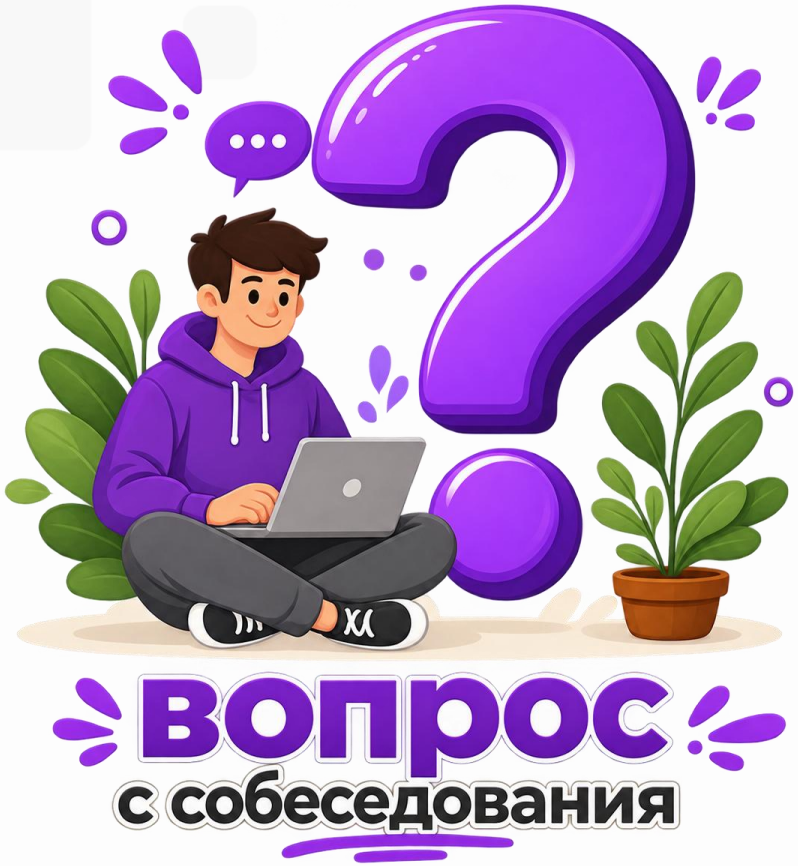
IThub

```
LinkedList elements size = 7. Elements:
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday

Remove "lalala" result = false
Remove "Wednesday" result = true
LinkedList elements size after remove = 6. Elements:
Monday
Tuesday
Thursday
Friday
Saturday
Sunday

Tuesday
```

IThub



Чем отличается ArrayList от LinkedList?

ArrayList

`[Alex] [Maria] [John] [Peter]`

Элементы находятся в массиве.

LinkedList

`[Alex] → [Maria] → [John] → [Peter]`

Каждый элемент связан с соседними.

Интерфейс Queue

IThub

```
interface Queue<E> extends Collection<E>
```

- обеспечивает поведение очереди, которая часто представляет собой список, действующий по принципу **“первым пришел - первым обслужен”**

Модификатор и тип	Метод	Описание
E	element()	Возвращает элемент из головы очереди, не удаляя его. Если очередь пуста, тогда генерируется исключение NoSuchElementException
boolean	offer(E e)	Предпринимает попытку добавления obj в очередь. Возвращает true, если объект e был добавлен, или false в противном случае
E	peek()	Возвращает элемент из головы очереди, не удаляя его. Если очередь пуста, тогда возвращается null
E	poll()	Возвращает элемент из головы очереди, в процессе удаляя его. Если очередь пуста, тогда возвращается null
E	remove()	Удаляет элемент из головы очереди, возвращая его. Если очередь пуста, тогда генерируется исключение NoSuchElementException

Интерфейс Deque

IThub

```
interface Deque<E> extends Queue<E>
```

- обеспечивает поведение двусторонней очереди
- можно добавлять и удалять элементы с обоих концов

Модификатор и тип	Метод	Описание
void	addFirst(E e)	Вставляет указанный элемент **в голову** двустронней очереди, выдает исключение IllegalStateException, если если в очереди с ограниченной емкостью отсутствует свободное пространство
void	addLast(E e)	Вставляет указанный элемент **в хвост** данной очереди, выдает исключение IllegalStateException, если в данный момент нет свободного места
E	getFirst()	Извлекает, но не удаляет первый элемент очереди
E	getLast()	Извлекает, но не удаляет последний элемент очереди
boolean	offer(E e)	Вставляет указанный элемент в хвост очереди и возвращает управление true в случае успешного выполнения и false, если в данный момент нет свободного места
E	peek()	Извлекает, но не удаляет, голову очереди или возвращает null, если очередь пуста
E	poll()	Извлекает и удаляет голову очереди или возвращает null, если очередь пуста
E	pop()	Возвращает элемент из головы двусторонней очереди, в процессе удаляя его. Если двусторонняя очередь пуста, тогда генерируется исключение NoSuchElementException
void	push(E e)	Добавляет объект e в голову двусторонней очереди. Генерирует исключение IllegalStateException, если в двусторонней очереди с ограниченной емкостью отсутствует свободное пространство
E	removeFirst()	Извлекает и удаляет первый элемент (голову) этой очереди. Если двусторонняя очередь пуста, тогда генерируется исключение NoSuchElementException
E	removeLast()	Извлекает и удаляет последний элемент этой очереди. Если двусторонняя очередь пуста, тогда генерируется исключение NoSuchElementException
E	peek()	Извлекает, но не удаляет, голову очереди или возвращает null, если очередь пуста
E	poll()	Извлекает и удаляет голову очереди или возвращает null, если очередь пуста
E	pop()	Возвращает элемент из головы двусторонней очереди, в процессе удаляя его. Если двусторонняя очередь пуста, тогда генерируется исключение NoSuchElementException

класс ArrayDeque

IThub

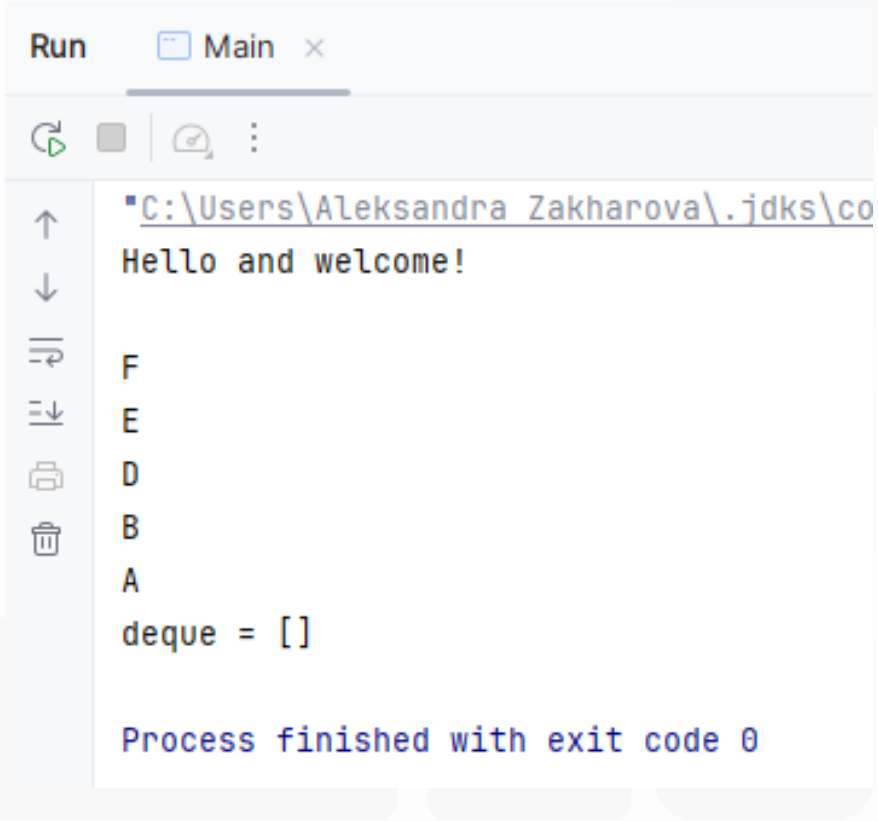
```
class ArrayDeque<E> extends AbstractCollection<E> implements Deque<E>
```

- создает динамический массив и не имеет ограничений по емкости
- емкость увеличивается по мере необходимости для обработки элементов, добавленных в двустороннюю очередь

```
public void arrayDequeDemo() {
    ArrayDeque<String> deque = new ArrayDeque<>();

    // добавляем объекты e в голову двусторонней очереди
    deque.push("A");
    deque.push("B");
    deque.push("D");
    deque.push("E");
    deque.push("F");

    while (deque.peek() != null) {
        System.out.println(deque.pop());
    }
    System.out.println("deque = " + deque);
}
```



класс PriorityQueue

IThub

```
class PriorityQueue<E> extends AbstractQueue<E>
```

- в E указывается тип объектов, которые будет хранить очередь
- создает очередь с приоритетом на основе компаратора очереди
- если при создании экземпляра PriorityQueue компаратор не указан, тогда применяется стандартный компаратор для типа данных, хранящихся в очереди. Стандартный компаратор упорядочивает очередь в порядке возрастания. Таким образом, в голове очереди будет наименьшее значение. Однако за счет предоставления специального компаратора появляется возможность указания другой схемы упорядочения

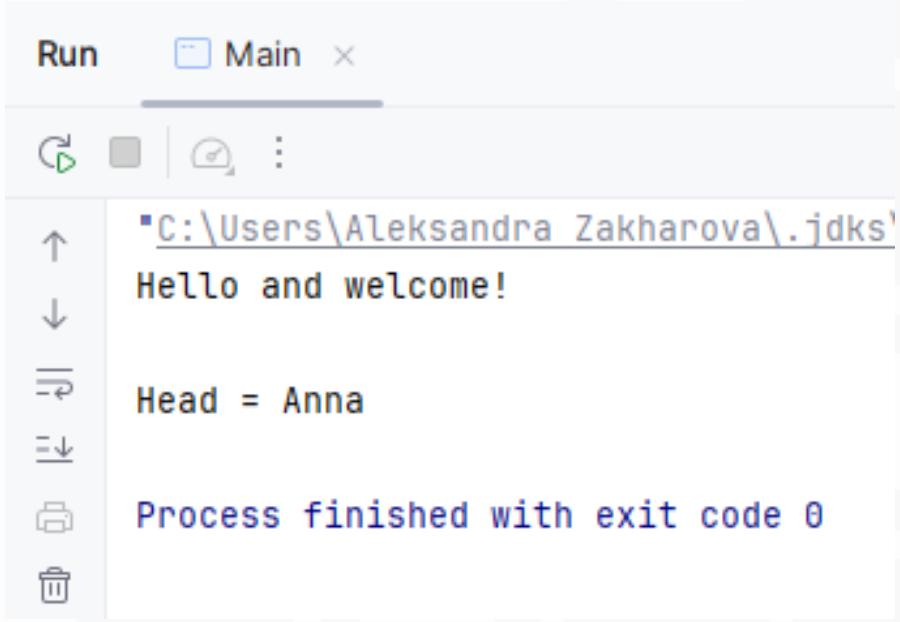
! эта реализация **не синхронизирована** (несколько потоков не должны обращаться к PriorityQueue экземпляру одновременно, если какой-либо из них изменяет очередь - вместо этого нужно использовать потокобезопасный PriorityBlockingQueue класс)

IThub

```java
public void priorityQueueDemo() {
    Comparator<String> comparator = Comparator.comparing(String::length);

    PriorityQueue<String> queue = new PriorityQueue<>(comparator);
    queue.add("Sasha");
    queue.add("Anastasia");
    queue.add("Anna");
    queue.add("Marina");

    System.out.println("Head = " + queue.peek());
}
```



Задача (ArrayDeque, PriorityQueue)

Необходимо разработать простую систему управления задачами.
У каждой задачи есть название и приоритет:

```
class Task {
    private String name;
    private int priority;
```

- Для хранения обычных задач используйте ArrayDeque<Task>
- Для хранения срочных задач используйте PriorityQueue<Task>
- При создании PriorityQueue необходимо передать **собственный Comparator**<Task>. Компаратор должен определять порядок задач по полю priority:
  1 — самый высокий приоритет
  2 — средний
  3 — низкий

то есть задача с приоритетом 1 должна извлекаться раньше задачи с приоритетом 2.

```
Comparator<Task> comparator = ...;

PriorityQueue<Task> urgentTasks =
        new PriorityQueue<>(comparator);
```

IThub

Создайте класс TaskManager. В нём должны находиться:

- private ArrayDeque<Task> normalTasks
- private PriorityQueue<Task> urgentTasks

Реализуйте методы:

- void addNormalTask(Task task) - добавляет обычную задачу
- void addUrgentTask(Task task)- добавляет срочную задачу.
- Task getNextTask() - возвращает следующую задачу для выполнения

Правила (метод getNextTask()):

- Если есть срочные задачи — сначала взять задачу из PriorityQueue.
- Если срочных задач нет — взять задачу из ArrayDeque.
- Если задач нет вообще — вернуть null.

```
manager.addNormalTask(new Task("Написать отчёт", 0));
manager.addNormalTask(new Task("Проверить почту", 0));
manager.addUrgentTask(new Task("Исправить баг", 2));
manager.addUrgentTask(new Task("Падение сервера", 1));
manager.addUrgentTask(new Task("Обновить документацию", 3));
```

должен получиться порядок:

Падение сервера
Исправить баг
Обновить документацию
Написать отчёт
Проверить почту

Интерфейс Set

```
interface Set<E> extends Collection<E>
```

Хранит уникальные элементы

```
static<E> Set<E> copyOf(Collection <? extends E> from)
```

возвращает набор, который содержит те же элементы, что и в коллекции from. Значения null не разрешены. Возвращаемый набор является немодифицируемым и основанным на значении

IThub

Класс HashSet

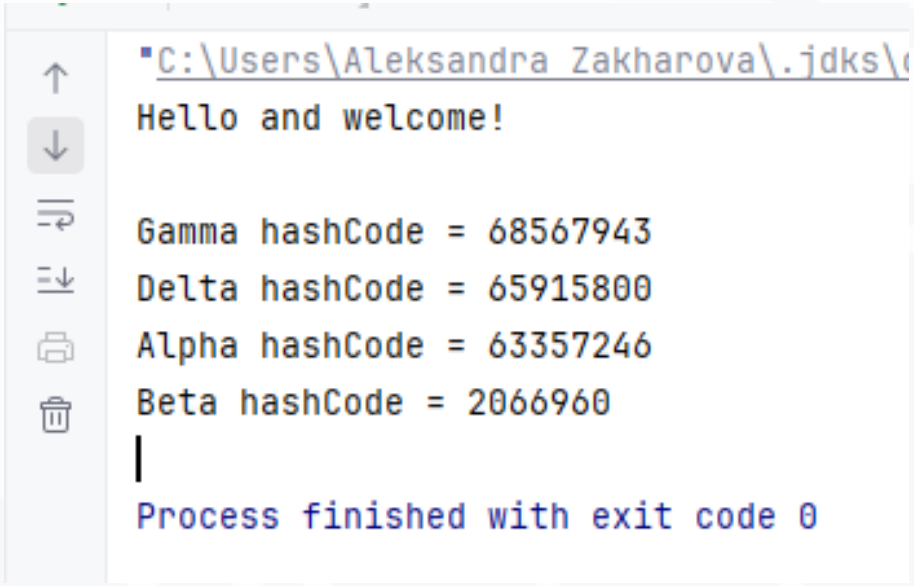
IThub

- создает коллекцию, в которой для хранения данных применяется хеш-таблица
- использует hash-код объекта для эффективного поиска

! эта реализация **не синхронизирована** (если несколько потоков одновременно обращаются к набору хешей, и хотя бы один из них изменяет этот набор, необходимо выполнить внешнюю синхронизацию)

```
public void hashSetDemo() {
    HashSet<String> set = new HashSet<>();
    set.add("Beta");
    set.add("Gamma");
    set.add("Delta");
    set.add("Alpha");

    // элементы не хранятся в отсортированном порядке
    set.forEach(s -> System.out.println(s + " hashCode = " + s.hashCode()));
}
```



Класс LinkedHashSet

IThub

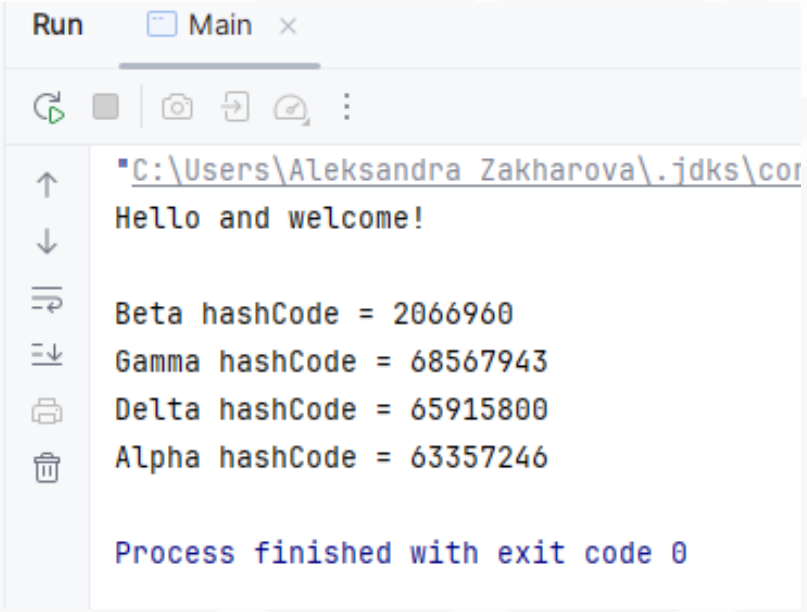
```
class LinkedHashSet<E> extends HashSet<E> implements Set<E>
```

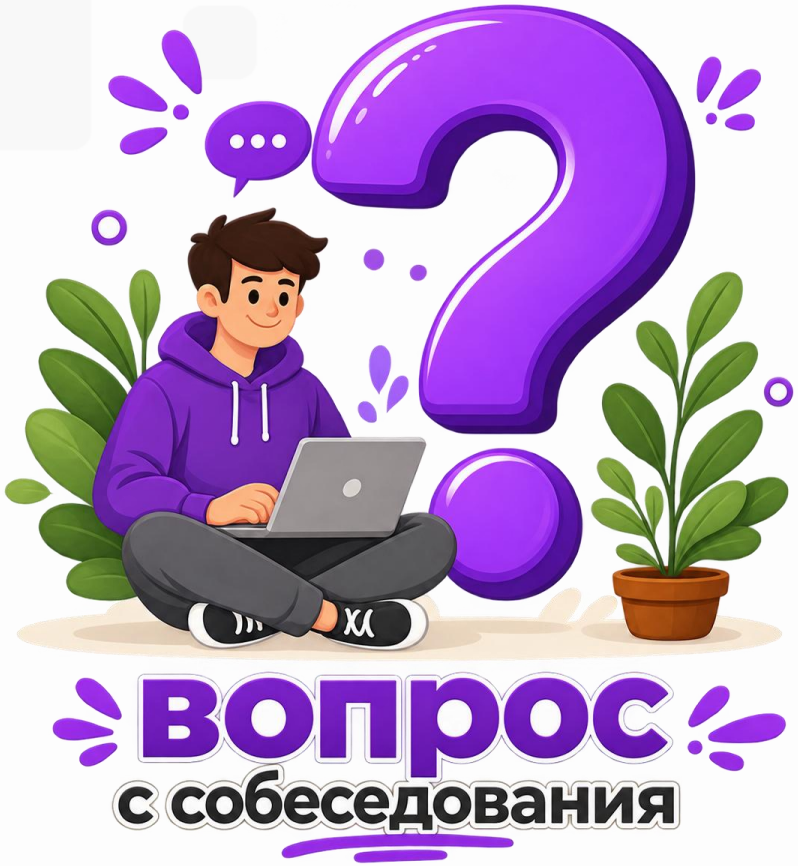
- поддерживает связный список элементов внутри набора **в порядке их добавления**

! эта реализация **не синхронизирована** (если несколько потоков одновременно обращаются к связанному набору хешей, и хотя бы один из них изменяет этот набор, необходимо выполнить внешнюю синхронизацию)

```
public void linkedHashSetDemo() {
    LinkedHashSet<String> set = new LinkedHashSet<>();
    set.add("Beta");
    set.add("Gamma");
    set.add("Delta");
    set.add("Alpha");

    // в порядке добавления
    set.forEach(s -> System.out.println(s + " hashCode = " + s.hashCode()));
}
```





Чем LinkedHashSet отличается от HashSet?

LinkedHashSet отличается от HashSet только тем, что в его основе лежит LinkedHashMap вместо HashMap и порядок элементов идентичен порядку добавления элементов.

Класс TreeSet

IThub

```
class TreeSet<E> extends AbstractSet<E> implements NavigableSet<E>
```

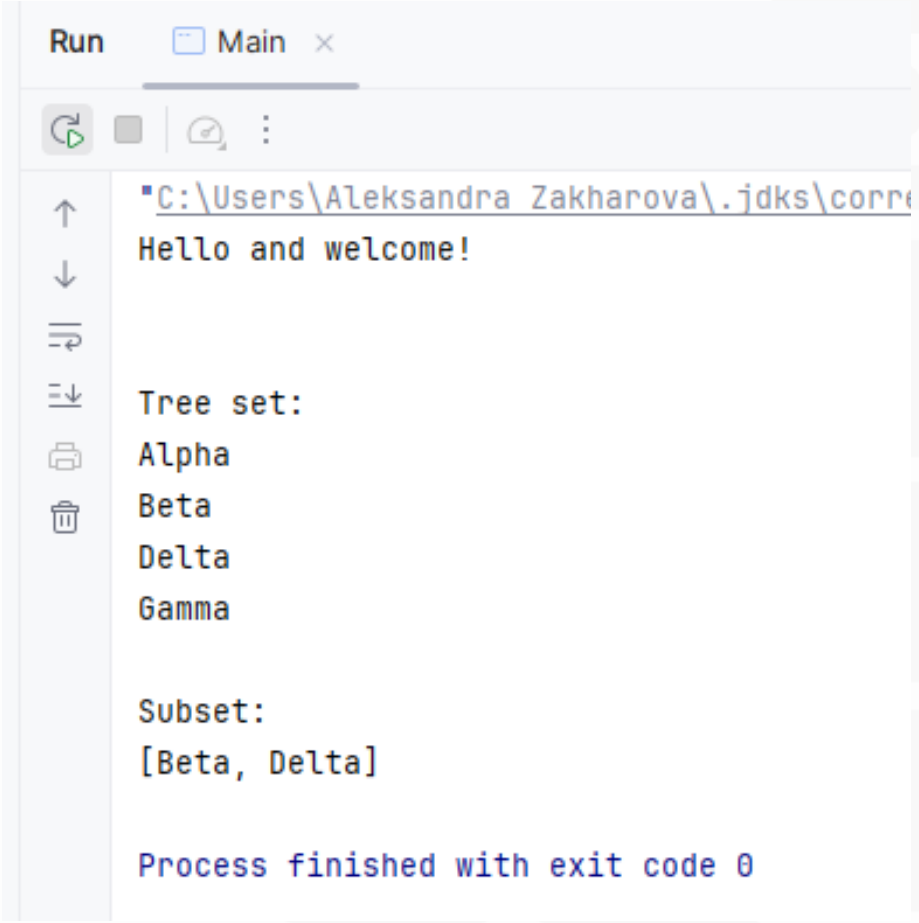
- Объекты хранятся **в отсортированном порядке по возрастанию**

! эта реализация **не синхронизирована** (если несколько потоков одновременно обращаются к набору, и хотя бы один из них изменяет набор, необходимо выполнить внешнюю синхронизацию)

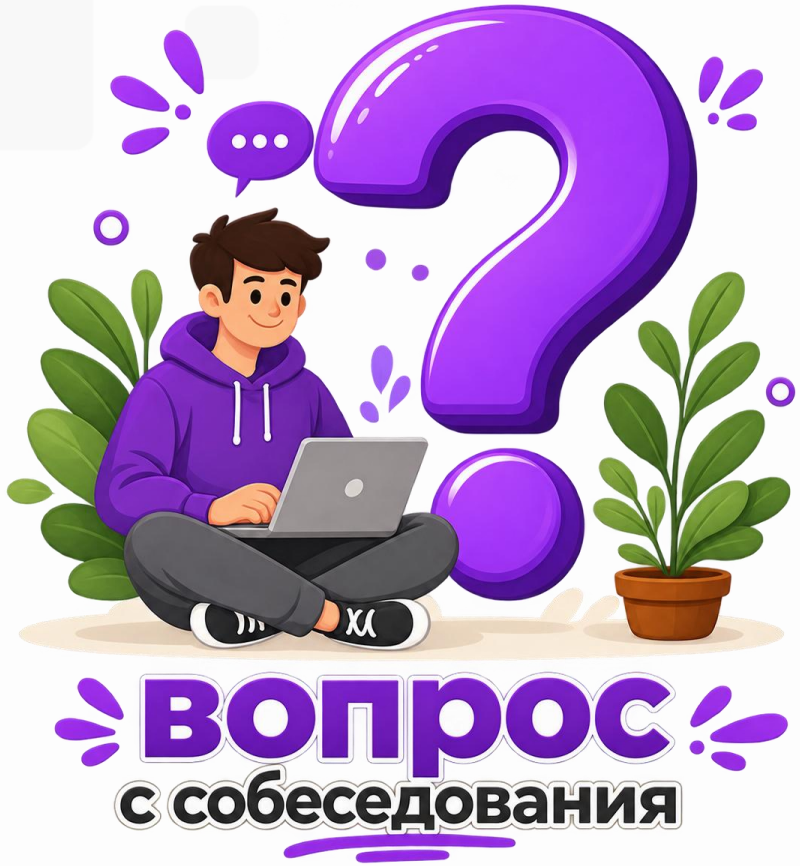
```java
public void treeSetDemo() {
    TreeSet<String> set = new TreeSet<>();
    System.out.println("\nTree set:");
    set.add("Beta");
    set.add("Gamma");
    set.add("Delta");
    set.add("Alpha");

    // отсортированный порядок
    set.forEach(System.out::println);

    System.out.println("\nSubset:");
    Set<String> subSet = set.subSet("Beta" , "Gamma");
    System.out.println(subSet);
}
```



IThub



В чем отличия TreeSet и HashSet?

TreeSet обеспечивает **упорядоченное** хранение элементов в виде красно-черного дерева.

HashSet **не** поддерживает **упорядоченное** хранение элементов.

Задача (HashSet, TreeSet, LinkedHashSet)

IThub

Разработайте программу для управления тегами статей в блоге.
Каждая статья содержит набор тегов (String).
Необходимо использовать три разных множества:

- HashSet — для быстрого хранения уникальных тегов
- LinkedHashSet — чтобы сохранять порядок добавления тегов
- TreeSet — чтобы хранить теги в отсортированном порядке

Создайте класс TagManager. В нём должны находиться:

- private HashSet<String> allTags;
- private LinkedHashSet<String> recentTags;
- private TreeSet<String> sortedTags;

Создайте метод **void addTag(String tag)**
При добавлении тега необходимо добавить его во все три множества. Попробуйте добавить какой-нибудь тэг 2 или 3 раза.

Создайте три метода:

- **void printAllTags()** - выводит теги из HashSet
- **void printTagsInAddOrder()** - выводит теги из LinkedHashSet
- **void printSortedTags()** - выводит теги из TreeSet

Добавьте метод **boolean containsTag(String tag)** - он должен возвращать true, если такой тег существует.